DOCUMENTO 02 · V1.1

# Plan de Desarrollo de Software

NexaSafe — Marco Scrum con modelo de entrega DevSecOps. Define el ciclo de vida, la organización del equipo, las ceremonias y la forma en que la seguridad se integra como parte innegociable del proceso.

| 22 sep 2026 | 3 oct 2026 | Scrum | 2 semanas | DevSecOps |
|---|---|---|---|---|
| FECHA DE ENTREGA (V1.0) | REVISIÓN V1.1 | MARCO ÁGIL | DURACIÓN DE SPRINT | MODELO DE ENTREGA |

---

## 1–2 Introducción y modelo de ciclo de vida

Este plan define cómo el equipo construirá NexaSafe entre el 22 de septiembre y el 28 de noviembre de 2026: modelo de ciclo de vida, organización, ceremonias, artefactos, criterios de entrada/salida y la forma en que DevSecOps se integra como parte no negociable del proceso.

| Scrum | 2 sem. | 2–3 | DevSecOps |
|---|---|---|---|
| MARCO ÁGIL | DURACIÓN DEL SPRINT | INTEGRANTES DEL EQUIPO | MODELO DE ENTREGA |

### 2.1 Modelo seleccionado: Scrum

Se adopta Scrum por tres factores del contexto: (1) los requisitos evolucionan a medida que se valida cada incremento con el docente y el "cliente simulado" del colegio; (2) el equipo es reducido (2–3 integrantes) y necesita ciclos cortos con retroalimentación frecuente; (3) la naturaleza sensible del dominio exige que cada incremento sea auditable y demostrable, lo cual encaja con la cadencia de Sprint Review.

### 2.2 Modelo de entrega: DevSecOps

Sobre Scrum se superpone un modelo de entrega DevSecOps con principio rector *shift-left*: la seguridad no es una fase previa a la entrega, es un criterio de la Definition of Done que se ejecuta automáticamente en cada commit. Desde el Sprint 0 existe pipeline de integración continua con controles de seguridad automatizados, antes de construir la primera historia de negocio.

### 2.3 Por qué no cascada ni Kanban puro

| MODELO | RAZÓN DE DESCARTE |
|---|---|
| **Cascada** | Exige requisitos cerrados desde el inicio; el proyecto depende de decisiones que solo se validan con el uso (p. ej. tolerancia del corredor seguro, umbral de sacudida para el pánico). |
| **Kanban puro** | No exige una revisión periódica formal con el Product Owner; en un entregable académico con fechas de sustentación fijas, el ritmo de Sprint Review/Retrospective da puntos de control necesarios. |

## 3 Organización del equipo

| ROL | RESPONSABLE | RESPONSABILIDADES |
|---|---|---|
| **Product Owner** | Docente del proyecto integrador (o representante del colegio como cliente simulado) | Prioriza el backlog, acepta o rechaza el incremento en la review, resuelve dudas de negocio. |
| **Scrum Master** | Integrante rotativo del equipo | Facilita ceremonias, retira impedimentos, protege el sprint de cambios de alcance. |
| **Development Team** | 3 integrantes: Móvil, Backend/Datos, DevSecOps/QA | Equipo multifuncional y autoorganizado. Cada integrante rota por el rol de Security Champion cada dos sprints. |

> **Ajuste por tamaño de equipo**
>
> El equipo es de 2 a 3 integrantes. Con 3, se mantiene la velocidad objetivo de 24 puntos por sprint. Con 2, se ajusta a 16–18 puntos por sprint, se combinan los roles de Dev y Scrum Master, y las historias `SHOULD` se sacrifican primero.

Cada historia de usuario es un issue en GitHub con plantilla que incluye título en formato de historia, criterios de aceptación en Gherkin, estimación en puntos y etiquetas (`epic:*`, `type:*`, `security`). La propiedad del código es colectiva: no hay conocimiento aislado en una sola persona (mitigación del riesgo R7).

## 4 Ceremonias y cadencia

Ciclo de ceremonias dentro de un sprint de dos semanas, en orden de ejecución:




↻ el ciclo se repite cada sprint de 2 semanas

| CEREMONIA | DURACIÓN | FRECUENCIA | SALIDA |
|---|---|---|---|
| **Sprint Planning** | 8 h | Inicio de sprint (lunes y martes, 4 h cada día) | Sprint Goal + Sprint Backlog comprometido |
| **Daily Scrum** | 15 min | Diario (asíncrono por Discord si no hay presencialidad) | Impedimentos identificados |
| **Refinement** | 1 h | Miércoles de la semana 1 | Historias del siguiente sprint estimadas y con DoR |
| **Sprint Review** | 4 h | Fin de sprint (viernes) | Incremento demostrado y feedback del PO |
| **Sprint Retrospective** | 3 h | Fin de sprint, después de la review | 1–2 acciones de mejora con responsable |
| **Security Review** | 30 min | Fin de sprint, previa a la review | Hallazgos del pipeline triados y priorizados en backlog |

### 4.3 Cadencia

- ✓ **Duración del sprint:** 2 semanas.
- ✓ **Sprint 0** dedicado a habilitación técnica y de seguridad (no produce valor de negocio, produce plataforma).
- ✓ **Sprint Goal único por sprint** , redactado en una sola frase orientada a resultado.

## 5 Artefactos Scrum

| ARTEFACTO | HERRAMIENTA | COMPROMISO ASOCIADO |
|---|---|---|
| **Product Backlog** | GitHub Projects (vista Backlog) | Product Goal |
| **Sprint Backlog** | GitHub Projects (vista Sprint actual: To Do / In Progress / In Review / Done) | Sprint Goal |
| **Incremento** | Build firmado en EAS + proyecto Supabase de staging migrado | Definition of Done |
| **Impediment Log** | Issues con etiqueta `impediment` | — |
| **Risk & Security Register** | `/docs/security/risk-register.md` versionado en el repo | — |

## 6 Definition of Ready y Definition of Done

### 6.1 Definition of Ready — para entrar al sprint

- ✓ Historia redactada en formato Como... quiero... para...
- ✓ Criterios de aceptación en Gherkin (Dado/Cuando/Entonces)
- ✓ Estimada por el equipo en puntos (Fibonacci)
- ✓ Dependencias técnicas identificadas y desbloqueadas
- ✓ Impacto en datos personales evaluado (¿toca datos de menores? ¿ubicación?)
- ✓ Diseño/wireframe disponible si la historia tiene UI
- ✓ Cabe holgadamente en un sprint (≤ 8 puntos; si no, se divide)

### 6.2 Definition of Done — para cerrar una historia

**FUNCIONAL**

- ✓ Todos los criterios de aceptación verificados
- ✓ Probada en dispositivo Android físico, no solo en emulador

**CALIDAD DE CÓDIGO**

- ✓ Pull Request revisado y aprobado por al menos un par
- ✓ Cobertura de pruebas unitarias ≥ 70 % en el módulo tocado
- ✓ Lint y formateo sin errores (ESLint + Prettier)
- ✓ Sin TODO ni código comentado en la rama principal

**SEGURIDAD — OBLIGATORIO**

- ✓ SAST (Semgrep) sin hallazgos High o Critical
- ✓ SCA (`npm audit --audit-level=high`) sin hallazgos High o Critical
- ✓ Escaneo de secretos (Gitleaks) limpio
- ✓ Ningún dato personal en logs (verificado manualmente en el PR)
- ✓ Tablas y Edge Functions nuevas con RLS, autorización por rol y rate limiting

**DOCUMENTACIÓN Y ENTREGA**

- ✓ README / CHANGELOG actualizados

- ✓ Tipos de Supabase regenerados y políticas RLS actualizadas si cambió el esquema
- ✓ Desplegado en el ambiente de staging por el pipeline
- ✓ Demostrable en la Sprint Review

## 7 Estrategia DevSecOps — resumen ejecutivo

El detalle completo del pipeline y el modelado de amenazas vive en el Documento de Diseño Arquitectónico. Este capítulo resume los elementos que afectan directamente la planificación del sprint.

### 7.1 Controles por fase del ciclo

| FASE | PRÁCTICA | HERRAMIENTA | CUÁNDO |
|---|---|---|---|
| **Plan** | Modelado de amenazas STRIDE por épica | OWASP Threat Dragon | Refinement |
| **Code** | GitHub Flow (`feature/*` → PR → `main`, sin rama `develop`), pre-commit hooks, revisión de pares obligatoria | pre-commit, Gitleaks, protección de `main` | Local / antes del merge |
| **Build** | SAST, SCA, SBOM | Semgrep, CodeQL, npm audit, Syft | En cada PR (SAST, SCA) / cierre (SBOM manual; automático desde la fase 2, E0-05) |
| **Test** | Unitarias, integración, E2E, DAST, MobSF | Jest, RNTL, Maestro, OWASP ZAP, MobSF | Cada PR / nocturno / prerelease |
| **Release** | Firma del artefacto, gestión de secretos | EAS Build + keystore, GitHub/EAS Secrets | Al taggear versión |
| **Deploy** | Verificación de políticas RLS antes de aplicar; despliegue a staging al fusionar en `main` | Supabase CLI + suite de pruebas de política | Antes del despliegue |
| **Operate / Monitor** | Logs sin PII, auditoría append-only, errores | JSON logging, tabla audit_log, Sentry | Runtime continuo |

### 7.2 Política de tratamiento de hallazgos

| SEVERIDAD | SLA DE CORRECCIÓN | EFECTO |
|---|---|---|
| CRITICAL | 24 h | Bloquea el merge y el despliegue |
| HIGH | Dentro del sprint actual | Bloquea el merge |
| MEDIUM | Siguiente sprint | Advertencia; entra al backlog como issue security |
| LOW | Backlog priorizado | Registrado, sin bloqueo |

> **Sin excepciones permanentes**
> Toda excepción se documenta en `/docs/security/exceptions.md` con justificación, responsable y fecha de vencimiento.

### 7.3 Rol de Security Champion

Un integrante del Development Team asume el rol por dos sprints: facilita el modelado de amenazas en el refinement, tría los hallazgos del pipeline antes de la Security Review, mantiene el registro de riesgos y las excepciones, y presenta el estado de seguridad en la Sprint Review (3 minutos fijos).

## 8 Gestión de riesgos del proyecto

| ID | RIESGO | PROB. | IMPACTO | RESPUESTA |
|---|---|---|---|---|
| R1 | La ubicación en segundo plano se detiene por optimización de batería del fabricante (Xiaomi, Huawei). | Alta | Alto | Investigación temprana en Sprint 2, antes de construir la ubicación en vivo (E4-02, Sprint 3); guía de exclusión de optimización; foreground service persistente. |
| R2 | Falsas alarmas frecuentes erosionan la confianza y la gente deja de atender. | Media | Alto | Corredor de tolerancia configurable, temporizador de confirmación, sin sanción por falsa alarma. |
| R3 | El colegio no designa responsable del puesto de control. | Media | Alto | Diseñar el flujo para que funcione solo con la red de apoyo si no hay institución activa. |
| R4 | Fuga de datos de menores. | Baja | Crítico | Cifrado, minimización, retención corta, RBAC, auditoría; pentest interno en la fase 2 (posterior al semestre). |
| R5 | El número de prueba de WhatsApp Cloud API solo alcanza 5 destinatarios verificados. | Alta | Medio | Restricción de diseño (máx. 5 contactos de seguridad); el push permanece como canal principal sin ese límite. |
| R6 | Alcance excesivo para el tiempo disponible del semestre. | Alta | Alto | MoSCoW estricto; las historias COULD se sacrifican primero; revisión de alcance en el hito H3. |

| ID | RIESGO | PROB. | IMPACTO | RESPUESTA |
|---|---|---|---|---|
| R7 | Rotación o baja disponibilidad de un integrante. | Media | Medio | Propiedad colectiva del código, documentación en el repo, sin conocimiento aislado en una persona. |
| R8 | Falsos positivos del disparo por sacudida (correr, bus, guardar el celular). | Alta | Medio | Aplica a la fase 2 (E6-02). Umbral calibrado + ventana de cancelación por PIN; se activa solo en modo trayecto; degradar a COULD si no es viable. |

# 9 Métricas de seguimiento

## 9.1 Métricas Scrum

| MÉTRICA | OBJETIVO | CÓMO SE MIDE |
|---|---|---|
| Velocidad | Estable ±15 % entre sprints | Puntos completados por sprint |
| Cumplimiento del Sprint Goal | ≥ 85 % de los sprints | Aceptación en la review |
| Trabajo no planificado | ≤ 15 % del sprint | Puntos de issues agregados en curso |
| Burndown | Descendente sin mesetas prolongadas | GitHub Projects |

## 9.2 Métricas DevOps (DORA)

| MÉTRICA | OBJETIVO |
|---|---|
| Frecuencia de despliegue a staging | ≥ 1 por día hábil |
| Lead time (commit → staging) | < 30 min |
| Tasa de fallo en cambios | < 15 % |
| Tiempo de recuperación (MTTR) | < 2 h |

## 9.3 Métricas de seguridad

| MÉTRICA | OBJETIVO |
|---|---|
| Vulnerabilidades Critical/High abiertas al cierre del sprint | 0 |
| Tiempo medio de remediación de hallazgos High | < 5 días |
| Cobertura del pipeline de seguridad | 100 % de los PR |
| Secretos filtrados detectados en main | 0 |

## 10 Plan de comunicación

| CANAL | PROPÓSITO | FRECUENCIA |
|---|---|---|
| **Discord del equipo** | Daily Scrum asíncrono, coordinación rápida | Diaria |
| **GitHub Projects** | Estado del Sprint Backlog y del Product Backlog | Continua |
| **GitHub Issues / PR** | Discusión técnica trazable de cada historia | Continua |
| **Reunión de Sprint Review** | Demostración del incremento y feedback del PO | Cada 2 semanas |
| **Acta de retrospectiva** | Registro de acciones de mejora con responsable | Cada 2 semanas |